

Advanced Programming

More monads

Fritz Henglein

Datalogisk Institut, Københavns Universitet (DIKU)

Goals

Week 3 (last week):

- monadic parsing; parser combinators; applicative parsing;
- parsing of the Arithmetic Programming Language (APL).

Week 4 (this week):

- computations as monadic values
- monad abstraction:
 - monad-parameterized code
 - free monads
- free monads in action

Next week: property-based testing.

Resources

- Course notes, chapter 4.
 - Some differences of names used in slides and notes.
- Monads.hs: every definition shown here, verbatim, compiling under -Wall. Schematic signatures, laws, counterexamples and A4 fragments are not definitions; where useful they appear there as comments.

Monads in Haskell

- Functor: a *type function* $f :: \text{Type} \rightarrow \text{Type}$ with function

```

1 class Functor f where
2   fmap :: (a -> b) -> (f a -> f b)
3   -- fmap id = id;  fmap (g . f) = fmap g . fmap f

```

- Monad: a *functor* $m :: \text{Type} \rightarrow \text{Type}$ with functions

```

1 class Functor m => Monad m where
2   return :: a -> m a
3   join   :: m (m a) -> m a
4   -- join (return c)           = c
5   -- join (fmap return c) = c
6   -- join (join c)           = join (fmap join c)

```

- Equivalently, with $>=>$ instead of `join`; and $>=>$, *Kleisli composition*, on top of either:

```

1 join c      = c >=> id
2 m >=> f      = join (fmap f m)
3 (f >=> g) x = f x >=> g

```

Monads for compositional side-effecting programming

- Executing code has generally two effects:
 - Returning a *value*: the value is used in subsequently executed code (main effect);
 - Side effects: other effects that may also impact the outcome of subsequently executed code, such as inputting or outputting data, updating memory, sending/receiving messages, etc.
- Conceptually, executing code consists of two phases:
 - 1 perform all side effects;
 - 2 return the computed value.
- Code may be
 - purely functional: no side effects (its total effect is described by the return value);
 - purely imperative: returns no value (its total effect is performed by side effects);
 - a mixture of both.
- Command/query separation design pattern: separate the basic interface to a stateful object into purely functional and purely imperative code.

Computations

- A *computation* is a *value* that wraps code that, *if and only if* executed, performs all its side effects and finally returns a value.
- Computations can be constructed *monadically*:
 - **type** $M\ a$: computations with side effects described by M and return values of type a .
 - `return v`: perform no side effects. Return value v .
 - `c >>= f`: first perform the side effects of c , then those of $f\ v$ where v is the value returned by c . Return the value returned by $f\ v$.
- Key points:
 - Computations themselves are *values* that can be stored in lists, passed to functions, etc.
 - `>>=` (or, equivalently, `join`) is the *only way of composing computations* in monadically constructed computations.
 - Composing computations does *not* execute the code in them.
 - Executing (“running”) a computation requires a function (usually called `run . . .`) that takes a computation as an argument and kicks off the execution.
 - Computations constructed monadically satisfy the monadic equalities.

Example: the purely functional state monad

- `FState s`: the prototypical computation – functionally update a state of type `s` (side effect), return a value of any type.
- A **newtype** is an *isomorphism*; record names for both directions:
`runFState . FState = id`, `FState . runFState = id`.

```
newtype FState s a = FState {runFState :: s -> (a, s)}
```

```
instance Monad (FState s) where  
  c >>= f = FState (\s -> let (a, s') = runFState c s  
                        in runFState (f a) s')
```

```
get :: FState s s  
get = FState (\s -> (s, s))  
put :: s -> FState s ()  
put s = FState (\_ -> ((), s))
```

- `FState` has previously been named `State`.
- We use `FState` since we will exhibit multiple state implementations.

Applicative and Functor for FState

```
-- newtype FState s a = FState {runFState :: s -> (a, s)}
```

```
instance Applicative (FState s) where
  pure a      = FState (\s -> (a, s))
  mf <*> ma = mf >>= (\f -> fmap f ma)
```

```
instance Functor (FState s) where
  fmap f m = m >>= (return . f)
```

- Neither definition says anything about FState. Haskell requires **Functor** and **Applicative** to be *declared*; for a monad they need not be *invented*, and these two lines recur verbatim for every monad here.
- pure is the one operation >>= cannot supply. It is return; Haskell requires the definition to sit here, and we write return everywhere else.
- fmap f m = m >>= (return . f) is a *monadic law*. The definition of fmap is forced by it.

Monads as abstract data types

- A *concrete monad* is a parameterized *abstract data type* $M\ a$ with the *monad interface*

```

1  type M a
2  fmap   :: (a -> b) -> (M a -> M b)
3  return :: a -> M a    -- also called pure
4  (>>=)  :: M a -> (a -> M b) -> M b  -- or join

```

satisfying the monad equalities (previous slide).

- It typically has *additional* operations, e.g. the state monad

```

1  type State s a
2  get      :: State s s
3  put      :: s -> State s ()
4  runState :: s -> State s a -> (a, s)

```

satisfying *additional* equalities, e.g.

```

1  put s >> get      = put s >> return s
2  put s >> put s'    = put s'
3  get >>= put        = return ()

```

Intermezzo: Observational equality

- What does $=$ in e.g. `put s >> put s' = put s'` mean?
- *Observational equivalence*: e and e' are *observationally equivalent* if $C[e] = C[e']$ for all contexts $C[]$ that have an observable type such as **Bool** or **Int**.
- *Observational equivalence depends on which operations are available in the context!!*
- Example

1 `put s >> get = return s      -- true or false?`

- Without function `runState`: True. In fact, all monadic values are observationally equivalent if there is no function that can observe the difference between them.
- With function `runState`: False. Let $C[] = \text{fst } (\text{runState } 99 \text{ (let } s = 7 \text{ in } [] \gg \text{get}))$:

1 `fst (runState 99 (let s = 7 in put s >> get >> get)) = 7`
 2 `fst (runState 99 (let s = 7 in return 7 >> get)) = 99`

Abstracting monads

- We have a purely functional implementation of the state monad,

1 `newtype FState s a = FState {runFState :: s -> (a, s)}`

- What if we want to change the implementation?
 - to eventually make it faster, or to put the state somewhere else (a file, a database, a device); but also
 - to keep the purely functional implementation for rapid development, stubbing, a reference implementation, and regression testing.
- Two ways: Monad parameterized code and free monads

Monad parameterized code: State monads

- Put the operations in a type class and quantify over the monad:

```

1  class (Monad m) => StateMonad m s | m -> s where
2      get  :: m s
3      put  :: s -> m ()
  
```

- The functional dependency $m \rightarrow s$ says: a monad determines the type of state it carries.
- The functional state monad is an instance:

```

1  instance StateMonad (FState s) s where
2      get    = FState (\s -> (s, s))
3      put s = FState (\_ -> ((), s))
  
```

Monad-parameterized code: State monad

```
modify :: (StateMonad m s) => (s -> s) -> m ()
modify f = get >>= (put . f)
```

```
tick :: (StateMonad m Int) => m Int
tick = do { n <- get; put (n + 1); return n }
```

```
push :: (StateMonad m [a]) => a -> m ()
push x = modify (x :)
```

```
pop :: (StateMonad m [a]) => m (Maybe a)
pop = do
  xs <- get
  case xs of
    []      -> return Nothing
    (y : ys) -> do { put ys; return (Just y) }
```

- The monad is passed *implicitly* by declaring which monad the `m` is instantiated to. (See later.)

A state monad with imperative state update

- Keep the state in an updatable reference of type `IIORef s`.

```
newtype IState s a = IState {runIState :: IORef s -> IO a}
```

```
instance Monad (IState s) where
```

```
  c >>= f = IState (\ref -> do
                                a <- runIState c ref
                                runIState (f a) ref)
```

```
instance StateMonad (IState s) s where
```

```
  get    = IState readIORef
  put s = IState (\ref -> writeIORef ref s)
```

- `>>=` hands *the same* reference to both computations: the state is overwritten in place, not threaded through functionally.

Standard functor and applicative operations

```
instance Functor (IState s) where  
  fmap f m = m >>= (return . f)  
  
instance Applicative (IState s) where  
  pure a    = IState (\_ -> return a)  
  mf <*> ma = mf >>= (\f -> fmap f ma)
```

- `fmap` and `<*>` are the same two definitions as for `FState`, and will be for every monad in this lecture. Only `pure` and `>>=` say anything about `IState`.

Passing the state monad instance by type declaration

```
tickF :: FState Int Int  
tickF = tick
```

```
tickI :: IState Int Int  
tickI = tick
```

```
> runFState tickF 0  
(0,1)  
> newIORef 0 >>= runIState tickI  
0
```

- In the definition of `tickF`, a record containing the monad operation definitions of `FState Int` is *implicitly* passed to `tick`.
- In the definition of `tickI`, a record containing the monad operation definitions of `IState Int` is *implicitly* passed to `tick`.

From operations to continuation-passing operations

- Start from *any* monad with `get` and `put` – that is, any instance of the class. Give each operation its *continuation* as an argument:

```

1 putk :: (StateMonad m s) => s -> (() -> m a) -> m a
2 putk s = (put s >>=)
3 getk :: (StateMonad m s) => (s -> m a) -> m a
4 getk   = (get >>=)

```

- The operations are interdefinable:

```

1 put s = putk s return           put s >>= m = putk s m
2 get   = getk return             get   >>= m = getk m

```

- The left column is the right unit law $m \gg= \text{return} = m$. No implementation is used – only the monad laws and the two operations.

Uninterpreted operations by constructors

- Take `getk` and `putk` as *uninterpreted* constructors of a type `M` still to be defined:

```
1 Getk  :: (s -> M a) -> M a
2 Putk  :: s -> M a -> M a
```

Note: `() -> M a` is isomorphic to `M a`.

- `M a` is now nothing but the type these constructors build. Name it `FSM s a`:

```
1 data FSM s a where
2   Return :: a -> FSM s a
3   Getk   :: (s -> FSM s a) -> FSM s a
4   Putk   :: s -> FSM s a -> FSM s a
```

- data** ... **where** is *GADT syntax*: for a regular recursive type, notation only – it spells out each constructor's type and means what the Haskell 98 form means. It also admits a more specific result type, and existential variables; one existential appears later (`ErrorOp`).

FSM s as a monad

```
instance Monad (FSM s) where
  Return x >>= f = f x
  Getk k    >>= f = Getk (k >=> f)
  Putk s m >>= f = Putk s (m >>= f)

instance Functor (FSM s) where
  fmap f m = m >>= (return . f)

instance Applicative (FSM s) where
  pure      = Return
  mf <*> ma = mf >>= (\f -> fmap f ma)
```

- Each operation carries its continuation k , respectively m , so $\gg=$ composes f with it.
- fmap and <*> are defined *from* $\gg=$.
- pure is return ; Haskell requires its definition in **Applicative**.

FSM s as a state monad

instance StateMonad (FSM s) s **where**

get = Getk Return

put s = Putk s (Return ())

runFSM :: s -> FSM s a -> (a, s)

runFSM s (Return x) = (x, s)

runFSM s (Getk k) = runFSM s (k s)

runFSM _ (Putk s m) = runFSM s m

■ The continuation-passing equalities by calculation:

```

1  get >>= f    = Getk Return >>= f
2              = Getk (\s -> Return s >>= f) = Getk f
3  put s >>= f = Putk s (Return ()) >>= f    = Putk s (f ())

```

■ A third StateMonad instance: tick, push, pop run unchanged – now building data. runFSM 0 (tick :: FSM Int Int) gives (0,1).

Abstracting the recursive occurrences

- Argument types of the three constructors, writing X for FSM s a :

```

1 Return :: a          -> X
2 Getk   :: (s -> X)   -> X
3 Putk    :: s -> X     -> X
4      X  =  a  +  (s -> X)  +  (s, X)

```

- X occurs once in each operation's argument except in `Return`. Replace it by a parameter r ; collect the two operation summands:

```

1 data StateOp s r where
2   StateGet :: (s -> r) -> StateOp s r
3   StatePut :: s -> r -> StateOp s r
4   X      =  a  +  StateOp s X

```

- `StateOp s r` is *one layer* of syntax: an operation, with r marking the holes its subcomputations occupy.
- Only *argument* occurrences are abstracted; the result occurrence makes these constructors of X , and the recursion below restores it.

Tying the knot

data Free e a **where**

Pure :: a -> Free e a

Free :: e (Free e a) -> Free e a

type FreeState s a = Free (StateOp s) a

- Free e a is the fixed point of equation $r = a + e\ r$; that is $\text{Free } e\ a = a + e\ (\text{Free } e\ a)$, where $=$ is type isomorphism. Constructor by constructor:

- 1 Return x <-> Pure x
- 2 Getk k <-> Free (StateGet k)
- 3 Putk s m <-> Free (StatePut s m)

Where **Functor** *e* comes from

- FSM's `>>=` applied (`>>= f`) to every subcomputation inside an operation, leaving the operation's own data alone:

```
1 Getk k    >>= f = Getk (\s -> k s >>= f)
2 Putk s m >>= f = Putk s (m >>= f)
```

- In the abstracted layer that operation is exactly `fmap`:

```
1 instance Functor (StateOp s) where
2   fmap h (StateGet k)   = StateGet (h . k)
3   fmap h (StatePut s a) = StatePut s (h a)
```

- So **Functor** *e* is not a condition imposed to make `Free` typecheck. It is FSM's `bind`, factored out and named – which is also why these **Functor** instances are forced rather than chosen.

Free e is a monad

```
instance (Functor e) => Monad (Free e) where
```

```
  Pure x >>= f = f x
```

```
  Free g >>= f = Free (fmap (>>= f) g)
```

```
instance (Functor e) => Functor (Free e) where
```

```
  fmap f (Pure x) = Pure (f x)
```

```
  fmap f (Free g) = Free (fmap (fmap f) g)
```

```
instance (Functor e) => Applicative (Free e) where
```

```
  pure = Pure
```

```
  Pure f <*> m = fmap f m
```

```
  Free g <*> m = Free (fmap (<*> m) g)
```

- `Pure x >>= f`: nothing recorded yet, so continue with `f x`.
- `Free g >>= f`: push `(>>= f)` into the holes of the layer `g`.
- The two `fmaps` in the `Free` case differ: outer is `e`'s, inner `Free e`'s.

FreeState is a state monad

```
instance StateMonad (Free (StateOp s)) s where
```

```
  get    = Free (StateGet Pure)
```

```
  put s = Free (StatePut s (Pure ()))
```

```
tickFree :: FreeState Int Int
```

```
tickFree = tick
```

- One more instance of the class. `tick`, `push`, `pop` and `stackExample` are unchanged; at this type they build a term instead of a function.
- `FreeState s` is isomorphic to `FSM s`:

```
1 toFree :: FSM s a -> FreeState s a
```

```
2 toFree (Return x) = Pure x
```

```
3 toFree (Getk k)   = Free (StateGet (toFree . k))
```

```
4 toFree (Putk s m) = Free (StatePut s (toFree m))
```

`fromFree` is the same in reverse; the two compose to the identity.

The recipe, for any operation

- The same three steps for an arbitrary primitive operation:

```

1  op  :: T1 -> ... -> Tn -> M R           -- given
2
3  opk :: T1 -> ... -> Tn -> (R -> M a) -> M a  -- pass the
4                                           -- continuation
5  Op   :: T1 -> ... -> Tn -> (R -> r) -> F r    -- abstract M a
6                                           -- to r

```

- When $R = ()$, the continuation $() \rightarrow r$ collapses to r . That is how `Putk` lost its $()$, and why `StatePut` carries a plain r .
- `fmap` is then forced: r occurs only in the continuation, so `fmap` composes onto it.

What is an interpretation?

- An *interpretation* of the effects e in a monad m is a function

1 $h :: \text{forall } x. e \ x \rightarrow m \ x$

- One equation per operation; no equation for sequencing – h never sees `Free` or `Pure`.
- What **forall** x forbids: one function, used at every result type, so h cannot
 - inspect or apply anything to a value of type x ;
 - branch on what x is;
 - call a continuation $x \rightarrow$ and use the result at a known type.

Only: pass the continuation on unchanged.

- So h fixes the meaning of the operations and nothing else. Relation to *naturality*: last section.

Interpreting the free monad in a monad

```
interpret :: (Monad m, Functor e)
          => (forall x. e x -> m x)
          -> Free e a -> m a
interpret _ (Pure x) = return x
interpret h (Free g) = join (fmap (interpret h) (h g))
```

■ Follow the types in the Free case:

```
1 g                :: e (Free e a)
2 h g              :: m (Free e a)
3 fmap (interpret h) (h g) :: m (m a)
4 join (...)       :: m a
```

- join's job here: run the interpreted continuation after the interpreted operation. Equivalently `h g >=> interpret h`.
- `h` interprets primitive operations as monadic values in the target monad.
- The target monad provides their sequencing.

Interpreting state

```
stateOps :: (StateMonad m s) => StateOp s x -> m x
stateOps (StateGet k)    = fmap k get
stateOps (StatePut s x) = put s >> return x

runFreeState :: (StateMonad m s) => FreeState s a -> m a
runFreeState = interpret stateOps

runFreeStateF :: FreeState s a -> s -> (a, s)
runFreeStateF m = runFState (runFreeState m)

runFreeStateI :: FreeState s a -> IORef s -> IO a
runFreeStateI m = runIState (runFreeState m)
```

- Two equations fix the meaning of StateGet and StatePut; recursion, Pure and sequencing come from interpret.

Or: interpret directly, and fuse

```
runState :: s -> FreeState s a -> (a, s)
runState s (Pure x)                = (x, s)
runState s (Free (StateGet k))     = runState s (k s)
runState _ (Free (StatePut s' m)) = runState s' m
```

- This is the hand-written interpreter from the notes. Up to the order of the arguments it is *the same function* as `runFreeStateF` – with `interpret` and the `FState` instance inlined and the intermediate closures fused away.
- The hand-written interpreter was never a different technique. It is the general one, specialised.
- Having both is useful: the compositional one for clarity, the fused one for speed, and each as a test oracle for the other.

One computation, four runs

```

> runFState tickF 0                -- pure threading
(0,1)
> newIORef 0 >=> runIState tickI    -- a mutable IORef
0
> runFreeStateF tickFree 0         -- free, interpreted purely
(0,1)
> newIORef 0 >=> runFreeStateI tickFree -- free, in IO
0
> runState 0 tickFree              -- free, fused
(0,1)

```

- `tickF` and `tickI`: Opaque functions—they cannot be inspected, only executed.
- `tickFree`: a *data structure* that can be inspected by pattern matching, not only executed.

Free monads in action: Logging

```
data FibOp a = FibLog String a

instance Functor FibOp where
    fmap f (FibLog s c) = FibLog s (f c)      -- forced, as always

type FibM a = Free FibOp a
fibLog :: String -> FibM ()
fibLog s = Free (FibLog s (Pure ()))

fib :: Int -> FibM Int
fib 0 = return 1
fib 1 = return 1
fib n = do fibLog ("fib(" ++ show n ++ ")")
           x <- fib (n - 1); y <- fib (n - 2)
           return (x + y)
```

`fib` declares precisely which uninterpreted effects it may have. Not **IO**, “anything”.

Free monads: Multiple interpretations

```
pureFibM :: FibM a -> a           -- discard the log
pureFibM (Pure x)                  = x
pureFibM (Free (FibLog _ c)) = pureFibM c

ioFibM :: FibM a -> IO a           -- print the log
ioFibM (Pure x)                    = return x
ioFibM (Free (FibLog s c)) = do putStrLn s
                               ioFibM c

logFibM :: FibM a -> (a, [String]) -- collect the log
logFibM (Pure x)                  = (x, [])
logFibM (Free (FibLog s c)) =
  let (x, msgs) = logFibM c in (x, s : msgs)
```

- In production: log to a file, a daemon, a ring buffer. In tests: collect and assert on the list. `fib` does not change.
- **Watch the order:** `s : msgs`, not `msgs ++ [s]`. The latter reverses the log *and* costs quadratic time.

Free-monad typed computations as arguments:

Memoisation

`fib` is exponential. memoisation needs state – but the state is not *observable*; it only makes things faster. Let the *interpreter* decide whether to do it.

```
data FibOp a = FibLog String a
              | FibMemo Int (FibM Int) (Int -> a)
              -- key ^COMPUTATION ^continuation

type FibM a = Free FibOp a
instance Functor FibOp where
  fmap f (FibLog s c)    = FibLog s (f c)
  fmap f (FibMemo n m c) = FibMemo n m (f . c)

fibMemo :: Int -> FibM Int -> FibM Int
fibMemo n m = Free (FibMemo n m Pure)
```

`fibLog` is unchanged; the three interpreters each gain a `FibMemo` clause.

Example: Fibonacci with memoisation

```
fib :: Int -> FibM Int
fib 0 = return 1
fib 1 = return 1
fib n = fibMemo n (do
  fibLog ("fib(" ++ show n ++ ")")
  x <- fib (n - 1)
  y <- fib (n - 2)
  return (x + y))
```

- `fibMemo n` does not memoise anything. It *records* that the computation it wraps is `fib` called with `n`.
- `fib` is a recursive definition with uninterpreted functions; it says nothing about caches, tables or `IORefs`.

Example: Fibonacci interpreters

```
-- A: ignore the hint. The other two clauses are unchanged.
pureFibM (Free (FibMemo _ fn c)) = pureFibM (c (pureFibM fn))

-- B: actually memoise.
memoFibM :: FibM a -> a
memoFibM m = fst (run [] m)
  where
    run :: [(Int, Int)] -> FibM b -> (b, [(Int, Int)])
    run cache (Pure x) = (x, cache)
    run cache (Free (FibLog _ c)) = run cache c
    run cache (Free (FibMemo n fn c)) =
      case lookup n cache of
        Just x  -> run cache (c x)
        Nothing -> let (x, cache') = run cache fn
                    in run ((n, x) : cache') (c x)
```

```
> memoFibM (fib 30)  -- instant; pureFibM takes seconds
1346269
```

Free-monad typed arguments

- `FibMemo n m k` is not “do an effect, then continue”. It contains a *whole subcomputation* `m`. It lets an interpreter decide whether to run it or not.
- A `StateMonad`-style interface does not offer this since its primitive operations do not have free-monad typed arguments.
- Primitive operations with free-monad typed arguments provide the basis for implementing control flow operations; e.g.

```
1 TryCatchOp    (EvalM Val) (EvalM Val) (Val -> a)
2 TransactionOp (EvalM Val)              (Val -> a)
3 LoopOp        (EvalM Val)              (Val -> a)
```

The same shape: catch

```
data ErrorOp e a where
  ErrorThrow :: e -> ErrorOp e a
  ErrorCatch :: ErrorM e x -> (e -> ErrorM e x)
                    -> (x -> a) -> ErrorOp e a

instance Functor (ErrorOp e) where
  fmap _ (ErrorThrow e)      = ErrorThrow e
  fmap f (ErrorCatch m h c) = ErrorCatch m h (f . c)

type ErrorM e a = Free (ErrorOp e) a

throw :: e -> ErrorM e a
throw e = Free (ErrorThrow e)
catch :: ErrorM e a -> (e -> ErrorM e a) -> ErrorM e a
catch m h = Free (ErrorCatch m h Pure)
```

Scrutinee and handler are *computations* at their own result type x ; only the third argument is a continuation, which is all `fmap` can reach.

Interpreting catch

```
runError :: ErrorM e a -> Either e a
runError (Pure x)                = Right x
runError (Free (ErrorThrow e))   = Left e
runError (Free (ErrorCatch m h c)) =
  case runError m of
    Right x  -> runError (c x)
    Left err -> runError (h err >=> c)
```

- The interpreter runs m , *looks at the outcome*, and runs the handler only on failure. It is free to run m once, not at all, or – for a retry – twice.
- Either way the continuation c runs *after* the handling is over, which is what puts it outside the handler's scope.
- Compare: in A4, `TryCatchOp` carries two `EvalM Val` values plus a continuation. Same shape, with x fixed to `Val` instead of existential.

Why the branches must not sit at a

- Suppose we had written the branches at the parameter instead:

```
1 data ErrorOp e a = ... | ErrorCatch a (e -> a)
2 fmap f (ErrorCatch m h) = ErrorCatch (f m) (f . h)
```

It typechecks, and the **Functor** instance is lawful – but `fmap` now rewrites `m`, so `>>=` *distributes into the branches*:

```
1 (m 'catch' h) >>= k = (m >>= k) 'catch' (\e -> h e >>= k)
```

- Then `k` runs *inside* the handler: if the `catch` succeeds and `k` afterwards throws, the handler catches an exception raised after the `catch` had finished.

```
1 k v = if v == 1 then throw "E2" else return v
2 > runError (catch (return 1) (\_ -> return 99) >>= k)
3 Right 99                -- should be Left "E2"
```

- What prevents it is not that the branches are Free values – they are here too – but their *own* result type. (A4, question 3.)

Free monads as data

```
modifyEffects :: (Functor e, Functor h)
               => (e (Free e a) -> h (Free e a))
               -> Free e a -> Free h a
modifyEffects _ (Pure x) = Pure x
modifyEffects g (Free e) =
  Free (fmap (modifyEffects g) (g e))
```

- Rewrites every operation node, leaving the sequencing alone.
- Example: Rewrite every ReadOp so that it sees a modified environment.

```
1 localEnv :: (Env -> Env) -> EvalM a -> EvalM a
2 localEnv f = modifyEffects g
3   where g (ReadOp k) = ReadOp (k . f)
4         g op         = op
```

Free monads as data

```
relabelNaive :: (String -> String) -> FibM a -> FibM a
relabelNaive f = modifyEffects g
  where g (FibLog s c) = FibLog (f s) c
        g op           = op
```

```
> logFibM (relabelNaive (map succ) (fib 3))
(3,["fib(3)","fib(2)"])      -- nothing renamed!
```

- `modifyEffects` walks the *spine*. Every `FibLog` in `fib` sits inside the `FibM` argument of a `FibMemo`, which the spine walk never enters.

Free monads as data

- An operation carrying a computation must rewrite that computation too:

```
1 relabel :: (String -> String) -> FibM a -> FibM a
2 relabel _ (Pure x) = Pure x
3 relabel f (Free (FibLog s c)) =
4   Free (FibLog (f s) (relabel f c))
5 relabel f (Free (FibMemo n fn c)) =
6   Free (FibMemo n (relabel f fn) (relabel f . c))
```

```
1 > logFibM (relabel (map succ) (fib 3))
2 (3,["gjc")4*","gjc")3*"])
```

Straight-line computations

If every primitive monadic operation returns `()`, a computation is a *list* of operations.

```
data AuditOp a = Say String a | Charge Int a
```

```
instance Functor AuditOp where
```

```
  fmap f (Say s c)    = Say s (f c)
```

```
  fmap f (Charge n c) = Charge n (f c)
```

```
type AuditM a = Free AuditOp a
```

```
say :: String -> AuditM ()
```

```
say s = Free (Say s (Pure ()))
```

```
charge :: Int -> AuditM ()
```

```
charge n = Free (Charge n (Pure ()))
```

```
order :: AuditM ()
```

```
order = do say "open tab"; charge 40  
         charge 25; say "close tab"
```

Analysing free monads: The easy case

```
totalCost :: AuditM a -> Int           -- no execution
totalCost (Pure _)                     = 0
totalCost (Free (Say _ c))             = totalCost c
totalCost (Free (Charge n c))          = n + totalCost c

transcript :: AuditM a -> [String]     -- no execution
transcript (Pure _)                    = []
transcript (Free (Say s c))            = s : transcript c
transcript (Free (Charge _ c))         = transcript c
```

```
> (totalCost order, transcript order)
(65,["open tab","close tab"])
```

Analysing free monads: The hard case

- Add *one* operation that returns a value the computation branches on:

```
1 data AuditOp a = ... | AskBudget (Int -> a)
```

- `totalCost` can no longer be written. To get past `AskBudget k` one must *choose* an `Int` and run `k` on it. The computation is no longer a list; it is a tree with potentially unbounded fanout.
- Simple remedies: analyse the prefix up to the first such operation, or analyse one path at a time.
- This is the price of `>>=`: the continuation is (always) a Haskell function, and Haskell functions are opaque (cannot be analysed at run time).
- More advanced remedy: Employ free *arrows*.

Suspended computations as values

```
type EventName  = String
type EventValue = Int
type Event      = (EventName, EventValue)

data EventOp a
  = WaitFor EventName (EventValue -> a)
  | LogMsg String a

instance Functor EventOp where
  fmap f (WaitFor s c) = WaitFor s (f . c)
  fmap f (LogMsg s c)  = LogMsg s (f c)

type EventM a = Free EventOp a

waitFor :: EventName -> EventM EventValue
waitFor s = Free (WaitFor s Pure)
logMsg  :: String -> EventM ()
logMsg s = Free (LogMsg s (Pure ()))
```

Suspended computations as values

```
adder, multiplier :: EventM ()
adder = do
  logMsg "starting adder"
  x <- waitFor "add"; y <- waitFor "add"
  logMsg (unwords [show x, "+", show y, "=", show (x+y)])
multiplier = do
  logMsg "starting multiplier"
  x <- waitFor "mul"; y <- waitFor "mul"
  logMsg (unwords [show x, "*", show y, "=", show (x*y)])

stepUntilWait :: EventM a -> IO (EventM a)
stepUntilWait (Pure x) = return (Pure x)
stepUntilWait (Free (LogMsg s c)) = do putStrLn s
                                         stepUntilWait c
stepUntilWait w@(Free (WaitFor _ _)) = return w
```

Run computation as far as possible; when a value is needed that we do not have, *stop* and return the remaining computation as a value.

The pump

*-- Hand one event to one process: if it is waiting for exactly
-- this event, resume it and run on to its next wait.*

```
deliver :: Event -> EventM () -> IO (EventM ())  
deliver (name, val) (Free (WaitFor wanted c))  
    | wanted == name = stepUntilWait (c val)  
deliver _ p          = return p
```

```
runEventM :: [EventM ()] -> [Event] -> IO [EventM ()]  
runEventM ps []          = mapM stepUntilWait ps  
runEventM ps (e : es) = do  
    ps' <- mapM stepUntilWait ps  
    ps'' <- mapM (deliver e) ps'  
    runEventM ps'' es
```

- The state of each process *is* the suspended computation.
- Each event is delivered to each process. A process not waiting for a delivered event ignores it.

An event pump

```
> runEventM [adder, multiplier]
      [("add",1),("mul",2),("add",4),("mul",5)]
starting adder
starting multiplier
1 + 4 = 5
2 * 5 = 10
```

- Simulation of broadcast-based totally ordered messaging to a set of concurrent sequential processes in about twenty lines of code.
- The suspended processes are values: put them in a list, store them, resume them not at all, once, twice, etc.

Free monads: Bad worst-case time complexity

- In `Free g >>= f`, the bind is pushed *through the whole of g* by `fmap`.
- So a left-nested chain

```
1 ((m1 >>= f1) >>= f2) >>= f3 ...
```

re-traverses its prefix at every step: building an n -operation computation that way takes time *quadratic* in n .

- Right-nested chains – what **do** notation produces from straight-line code – are fine. Recursive functions that append are how you hit it.
- Counted: `StatePut` nodes rebuilt over a full run of n puts.

	n=4	n=8	n=16	n=100	
right-nested	4	8	16	100	-- n
left-nested	6	28	120	4950	-- $n(n-1)/2$

- Associativity of `>>=` still holds: both parenthesizations yield the *same term*. Only the cost of building it differs.
- Constant-time alternatives: last section.

Categorical notions

Not needed for A4; the source of the terminology.

- *Category*: objects, arrows, identities, associative composition. Hask: types and functions between them.
- *Functor* $F : \mathcal{C} \rightarrow \mathcal{C}$: a map on objects and a map on arrows, preserving identity and composition. In Haskell: the type constructor `plus` `fmap`, plus the two `fmap` laws.
- *Monad* on $\mathcal{C}$: a functor M with two natural transformations, *unit* (`return`) and *join*, satisfying

```

1  join (return c)      = c
2  join (fmap return c) = c
3  join (join c)        = join (fmap join c)

```

- The `>>=` form used throughout is interdefinable:

```

1  join c  = c >>= id           m >>= f = join (fmap f m)

```

- *Kleisli composition* (`>=>`) makes the three laws read as the category axioms: `return` is the identity, and `>=>` is associative.

Parametricity and naturality

- *Parametric polymorphism* is a property of $h :: \text{forall } x. e \ x \rightarrow m \ x$ as a *program*: one definition, used at every x , with no case analysis on x and no type-directed dispatch. This is what was exploited above.
- *Naturality* is a property of a *family* $h_A : FA \rightarrow GA$ of arrows: for every $g :: a \rightarrow b$ the square commutes,

$$1 \quad \text{fmap } g \ . \ h \ = \ h \ . \ \text{fmap } g$$

A family can be given ad hoc, one component at a time, and still be natural.

- Relation: for this type, naturality is a *consequence* of parametricity – an instance of Reynolds' abstraction theorem, in Wadler's reading, "theorems for free".
- Caveats in Haskell: `seq`, non-termination and `unsafeCoerce` invalidate the free theorem.

What “free” means

- Free e is the free monad on the functor e : left adjoint to the forgetful functor from monads to functors.
- Universal property: for every monad m and every $h :: \text{forall } x. e\ x \rightarrow m\ x$ there is *exactly one* monad morphism $\text{Free } e \rightarrow m$ extending h , namely $\text{interpret } h$.
 - *Existence*: nothing to decide beyond the meaning of each operation.
 - *Uniqueness*: fixing get and put fixes the meaning of every computation built from them.
- Consequence used above: $\text{Free } e$ imposes no equations of its own. In FreeState ,

1 `put s >> put s'    /=    put s'`

– two StatePut nodes against one. The equation holds only of the *interpretations*.

- So $\text{Free } (\text{StateOp } s)$ is the syntax of state operations; the state monad is one quotient of it.

Constant-time bind

The quadratic left-nesting above is a property of this representation, not of free monads.

- *Continuation-passing (Church/Codensity) encoding*, e.g. `Control.Monad.Free.Church`: `>>=` in constant time. No constructors remain to pattern match on, so static analysis and the rewriting of computations – the reason for using `Free` here – are lost.
- *Type-aligned catenable queue* for the continuation (van der Ploeg & Kiselyov, *Reflection without Remorse*, 2014): constant-time `>>=`, amortized constant-time inspection of the next operation, constructors retained.
- Same phenomenon as left-nested `++` on lists, and the same two families of fix: difference lists, or a catenable sequence.

Combining effects

- *Monad transformers* (`StateT`, `ExceptT`, ..., the `mtl` library): one monad is built by stacking layers, each adding an effect to the monad below, so the order of the stack is part of the semantics.
- *Algebraic effects and handlers* (Plotkin & Power; Plotkin & Pretnar; the languages `Eff` and `Koka`, and `OCaml 5`): operations are declared as a signature and given meaning by a handler installed around a computation – the separation made here by `Free` and `interpret`, but provided by the language.

Beyond monads

- *Applicative functors*: sequencing without data dependence between steps. The whole structure is static – the ceiling seen earlier does not arise.
- *Arrows*: composition of computations (`arr`, `>>>`, `first`) rather than of continuations. The continuation of a step need not be an opaque Haskell function, so more of a computation stays inspectable.
- *Free arrows*: the same construction one level up; the analogue of today's `Free`.
- *Monoidal and symmetric monoidal categories*: the setting in which arrows are the natural notion, as monads are for `>>=`.

Summary

- Side effects encapsulated as computation monads: different effects plugged in without changing source code.
- **Monad-parameterized code, and free monads:**
 - *Monad-parameterized code.* Pattern:
 $g :: (\text{Monad } m) \Rightarrow T \rightarrow m\ U$, operations in a type class.
 Simple. The program is a function: nothing to inspect, transform or resume.
 - *Free monads.* A functor F with one constructor per primitive operation;
 $\text{Free } F$; one $h :: \text{forall } x. F\ x \rightarrow M\ x$ per target. The computation is *data*: several interpretations, rewriting, analysis, suspension.
- $\text{Free } e$ imposes no equations: $\text{put } s \gg \text{put } s'$ and $\text{put } s'$ are different terms, equal only after interpretation.
- Effects with *computations* as arguments, not just continuations for advanced control flow and memory management
- **IO**, exceptions, `IOMRef`: notes, chapter 4.

On to assignment 4

- **Part A** – small, self-contained, all from this lecture:
 - derive an effect functor from a list of operations;
 - write three interpreters for one computation, and check they agree;
 - write the generic `interpret` and instantiate it twice;
 - an effect whose argument is a computation – and what breaks if it is not;
 - analyse a computation without running it, and find the ceiling.
- **Part B** – the APL evaluation monad, extended with
 - `TryCatchOp`: exceptions;
 - a key-value store, interpreted purely *and* against a database file;
 - interactive recovery from a missing key – an effect that only makes sense in one of the two interpreters;
 - `TransactionOp`: all-or-nothing writes, including nested;
 - `Break`: leaving a loop early – you design these effects yourself.
- Part A: warm-up, one to two hours. Part B: the assignment.